

GramVarKit: A Rule-Driven Toolkit Generator for Exploring AI-Oriented Grammar Variants

Cenyuan Zhang
Monash University
Australia
cenyuan.zhang@monash.edu

Zhensu Sun
Singapore Management University
Singapore
zssun@smu.edu.sg

Dangfeng Pan
Monash University
Australia
dangfeng.pan1@monash.edu

David Lo
Singapore Management University
Singapore
davidlo@smu.edu.sg

Xiaoning Du
Monash University
Australia
xiaoning.du@monash.edu

Abstract

AI-oriented grammar is a recently proposed concept aimed at enhancing the productivity of Large Language Models in processing source code. By redesigning programming language grammars to better suit LLMs, such grammars can enhance LLM performance from multiple aspects. For example, a recent paper improves token efficiency by representing programs with fewer tokens, thereby reducing the cost for LLMs to understand or generate source code. Although this research field holds substantial potential for future exploration, working with AI-oriented grammars necessitates not only grammar design but also a sophisticated toolkit for parsing and transforming code between the AI-oriented grammar and its original form. Implementing such a toolkit involves considerable effort, making it a barrier for future research. To address this, we implement GramVarKit, an open-source tool that automatically generates the toolkit for AI-oriented grammars using only a configuration file that describes the differences between two grammars. It provides a unified platform, enabling researchers to focus on advancing the design and application of AI-oriented grammars rather than the complexities of toolkit implementation.

Keywords

AI-oriented Grammar, Grammar-aware Transformation, Code Representation, Large Language Models

1 Introduction

Large Language Models (LLMs) [3, 6] and coding agents [1, 9] have demonstrated remarkable capabilities on code-related tasks and have become important producers and consumers of programs. However, current high-level programming languages are primarily designed for human developers and machine execution, and may not align well with how LLMs process and generate code. Human-centric features such as punctuation, indentation, and explicit delimiters help developers recognize program structure but may unnecessarily increase sequence length for LLMs, leading to higher computational costs and latency during inference.

This mismatch motivates the concept of AI-oriented grammar proposed by [10]. AI-oriented grammar seeks to derive variants of existing programming languages by redesigning their grammar to better suit LLMs while preserving their semantic equivalence. To demonstrate this concept, the authors developed SimPy, a simplified Python grammar, as a proof-of-concept. Their results show that

SimPy can reduce token counts by 8.6% to 34.7%, thereby saving LLMs’ computational cost during inference by a similar level[8].

Although SimPy demonstrates the feasibility of AI-oriented grammar, it represents only one manually crafted instance in the large design space and leaves many research questions unexplored. For instance, the impact of different grammatical elements on LLMs remains unclear, as does which forms of grammar are more comprehensible to models. Systematically investigating these questions requires a principled way to construct and compare diverse grammar variants. Since a grammar is essentially a set of production rules specifying all valid strings in a formal language, variants of an existing language can be obtained by applying grammar-aware edits to selected production rules while preserving the fundamental semantics. By composing different edits, researchers can derive a wide range of grammar variants and systematically evaluate how each design choice contributes to LLM performance.

However, exploring this design space at scale requires more than designing the grammars themselves. Each variant must be supported by bidirectional conversion between the original and the derived grammar. The forward direction, converting existing code into the derived syntax, is needed to construct datasets for evaluation or training. The backward direction, converting the derived syntax back to the original, serves two purposes: it helps identify grammar variants that are ambiguous or fail to preserve semantics, and it additionally supports the practical use of the new grammar, where human developers review and maintain ordinary source code while coding agents operate on the AI-oriented representation. Manually building such bidirectional tooling for every variant makes systematic exploration costly, repetitive, and error-prone.

A natural question is how such bidirectional tooling can be automatically built. Naive text-level replacement is insufficient: the same surface notation may appear in different grammar contexts and serve different syntactic functions, so transformations must be aware of the parse tree. Existing tools fall short of this need. Structural rewriting tools such as ast-grep [2] and Comby [5] support syntax-aware or template-based search and replacement within an existing language, but they do not derive a new grammar from production-rule edits, generate a parser for the derived grammar, or automatically maintain bidirectional converters. GrammarOptimizer [12] operates on grammar specifications without supporting the rewriting of existing code, and TAHR [4] is not tailored to rapidly exploring closely related variants derived from local edits

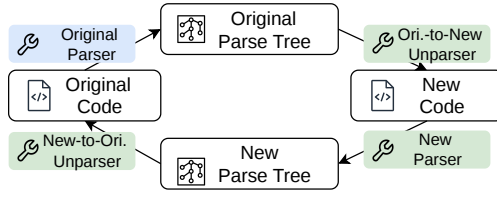


Figure 1: Tools required for the seamless transformation between standard programming language code and new code in AI-oriented grammar. The original parser, shown in blue, is reusable, while the derived parser and the two unparsers, shown in green, are variant-specific components. on an existing grammar. These limitations motivate a bidirectional parser-unparser architecture.

Figure 1 illustrates the bidirectional parser-unparser workflow for converting between the original and derived grammars. Here, a *parser* denotes a component that maps code to a parse tree and *unparser* denotes a component that reconstructs code from a parse tree. Supporting bidirectional conversion for each grammar variant therefore requires two parsers and two unparsers. While the original parser can often be reused from existing parser generators such as tree-sitter [11], the remaining components are variant-specific and costly to implement manually.

To fill this gap, we propose GramVarKit, a tree-sitter-based toolkit generator for AI-oriented grammar variants. GramVarKit targets variants that preserve the original AST structure, where syntax changes can be expressed as local node mappings over the parse tree. This design choice allows grammar transformation to be specified through a user-defined configuration, isolating the implementation as an automated and stable generation process from the specification. Users describe edits to selected production rules in a configuration file, from which GramVarKit automatically generates the derived grammar file and the two direction-specific unparsers. In this work, we instantiate GramVarKit for Python and Java.

To demonstrate GramVarKit in action, we take SimPy [10] as an example and generate its toolkit using GramVarKit. We evaluate the generated toolkit on two datasets, CodeSearchNet [7] and The Stack v2 [6]. The results confirm that the toolkit correctly transforms Python code into the SimPy syntax specified by the configuration file and converts it back to Python without loss of semantics. This demonstrates that GramVarKit can faithfully instantiate a given grammar-variant design from configuration rules.

2 The GramVarKit Tool

2.1 Overview

Figure 2 presents the system structure and workflow of GramVarKit. The tool takes a user-defined configuration file as input and generates the variant-specific components of a toolkit for a target AI-oriented grammar, while reusing shared components associated with the original language.

2.2 Supported Grammar Modifications

The syntax of a programming language can be defined using a set of production rules. Each production rule takes the form $A \rightarrow \alpha$, where A is a non-terminal symbol, referred to as the *head* of the

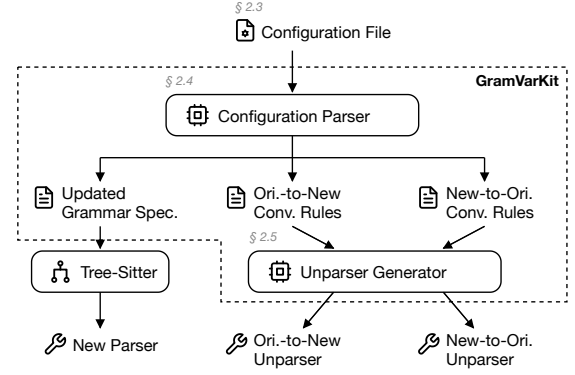


Figure 2: An overview of GramVarKit.

rule, and α is the *body* of the rule, consisting of an ordered sequence of terminal and non-terminal symbols. For example, the Python `if_statement` rule in tree-sitter can be written as:

```
1 if_statement: $ => seq(
2   'if', field('condition', $.expression),
3   ':', field('consequence', $_.suite),
4   repeat(field('alternative', $.elif_clause)),
5   optional(field('alternative', $.else_clause))
6 )
```

GramVarKit targets AI-oriented grammar variants that preserve the AST structure of the original language. Under this scope, the derived grammar keeps the same parent-child organization of existing nonterminal structures: non-terminals are not split, merged, or structurally reorganized. In terms of production rules, this means that modifications are applied to terminal symbols in selected fragments of production bodies. We refer to these modifications as *local edits*: they replace, delete, or insert symbols within such fragments. For instance, in the `if_statement` production, a supported modification may replace the keyword 'if' or delete the delimiter ':'. Multiple local edits may be applied to different fragments of the same production rule, and edits across multiple production rules together define the complete variant. This allows a grammar variant to be specified as a collection of production-body edits encoded in a configuration file. For modifications that fall outside the built-in local edits, GramVarKit provides an extension point through user-defined grammar modifications. For example, rewriting an `elif_clause` as a nested `else_clause` containing another `if_statement` is not a local edit as it reorganizes non-terminal structures.

2.3 Configuration File

The configuration file is the user-facing abstraction for specifying the local edits described in §2.2. It follows a JSON format and is centered on a list of *transformation rules*.

Each rule is associated with one production rule through the *prod_rule* attribute and describes how a fragment of its body is changed. The attributes *original* and *new* specify the corresponding fragments before and after the transformation, each represented as a contiguous ordered sequence of symbols.

Listing 1 shows an example transformation rule for the Python `if_statement` production introduced above. It rewrites the prefix

ending at the consequence field, while the remaining `elif_clauses` and optional `else_clause` are omitted because they are unchanged.

Listing 1: A configuration file with an example transformation rule.

```

1  [..., {
2    "prod_rule": "if_statement",
3    "original": [
4      {"type": "if"},
5      {"field": "condition", "anchor": true},
6      ":",
7      {"field": "consequence", "anchor": true}],
8    "new": [
9      {"type": "<if_stmt>"},
10     {"field": "condition", "anchor": true},
11     {"field": "consequence", "anchor": true}]
12  }, ...]
```

Symbols in *original* and *new* may be identified either by tree-sitter node *type*, or by tree-sitter's named child *field*, which captures the symbol's semantic role within the production.

Symbols that appear only in *original* or only in *new* are interpreted as the parts changed by the transformation, corresponding to deletion, insertion, or replacement. An *anchor* is a preserved symbol marked in both *original* and *new*, indicating that the two occurrences correspond to the same symbol before and after the transformation. Anchors provide reference points between *original* and *new* fragments. By comparing the regions before, between, and after anchors, GramVarKit infers the intended edits.

2.4 Configuration Parser

The configuration file is processed by a configuration parser, which produces two outputs: an updated grammar specification that is passed to tree-sitter for generating the parser of the new grammar, and two sets of *conversion rules* that are later consumed by the unparser generator (§2.5), one set for each direction.

A transformation rule in the configuration file is *declarative*: it specifies how two grammar fragments correspond to each other, leaving the underlying edits implicit. A conversion rule, in contrast, is *operational*: it specifies when and how a concrete edit should be applied during grammar updating or unparsing. This separation allows users to describe most grammar variants at the level of production-body fragments, while GramVarKit compiles them into the lower-level rules required by the implementation.

Each conversion rule contains a *condition* and an *action*. The *condition* describes the parse-tree context in which an edit should be applied, such as the parent node type, current node type or field, and neighboring siblings. It ensures that an edit is triggered only at the intended occurrence, which is necessary because the same surface symbol may appear in different grammar contexts. The *action* specifies the edit to be performed—insertion, deletion, replacement, or invoking a user-defined conversion function (§2.5). Optional *content* and *params* attributes carry the data needed by the action, such as the inserted text or the keyword arguments passed to a user-defined function.

Listing 2: Ori-to-new conversion rules derived from Listing 1.

```

1  [{"condition": {
2    "parent_type": "if_statement", "type": "if",
3    "next_sibling_field": "condition"},
4    "action": "replace", "content": "<if_stmt>"}],
```

```

5  {"condition": {
6    "parent_type": "if_statement", "type": ":",
7    "prev_sibling_field": "condition",
8    "next_sibling_field": "consequence"},
9    "action": "delete"}]
```

Listing 2 shows the two ori-to-new conversion rules derived from the transformation rule in Listing 1. Here, the anchor information is compiled into contextual constraints in the *condition* field, ensuring that the replacement and deletion apply only at the intended positions. The new-to-ori rule set is derived analogously by swapping the roles of the *original* and *new* fragments, although the resulting conversion rules are not necessarily in one-to-one correspondence.

The conversion rules are also used to derive the updated grammar specification. For each transformation rule, the configuration parser locates the corresponding production in the source-language grammar and applies the ori-to-new edits to its body. The resulting grammar specification is passed to tree-sitter to generate a parser for the target grammar variant.

2.5 Unparser Generator

The unparser generator produces, for each grammar variant, an ori-to-new unparser and a new-to-ori unparser by specializing a shared *base unparser*, a language-specific component that reconstructs source code from a parse tree of the source language. The base unparser handles language-level concerns. For example, Python requires special handling of indentation, newlines, and block structure, whereas Java relies on braces and semicolons to delimit blocks and statements. This design separates language-specific unparsing from variant-specific grammar transformation. Supporting a new language mainly requires a tree-sitter grammar and a base unparser for that language. Once this base component is available, multiple variants of the same language can be instantiated mostly through configuration files.

2.5.1 Ori-to-new unparser. For ori-to-new conversion, GramVarKit first parses the input program with the original parser of the source language. The unparser then traverses the resulting parse tree and, at each node, checks whether any ori-to-new conversion rule matches the node and its context. When a rule is triggered, its action modifies the emitted code fragment by inserting, replacing, or deleting text.

Most local syntax changes are handled by these built-in edit actions, while transformations that cannot be expressed as local edits are supported through user-defined conversion functions, which are invoked during unparsing with the signature

```
func(self: Unparser, node: Node, **kwargs) -> None
```

where *self* is the unparser instance, *node* is the current tree-sitter node, and the remaining keyword arguments are supplied by the *params* attribute of the conversion rule.

2.5.2 New-to-ori unparser. For new-to-ori conversion, GramVarKit first parses the input program with the generated parser for the target grammar. Because the resulting parse tree may contain target-specific tokens or omit tokens required by the original syntax, GramVarKit wraps it in an intermediate representation, the *wrapper*, before unparsing. The wrapper applies new-to-ori conversion

Table 1: Parse tree consistency and average time consumption on two datasets.

Language	Dataset	Consistency	Time per conversion
Python	CodeSearchNet	100%	6ms
	the Stack v2	100%	35ms
Java	CodeSearchNet	100%	3ms
	the Stack v2	100%	17ms

rules to adjust the tree structure—inserting omitted tokens, deleting target-specific tokens, or replacing nodes with their source-language counterparts—without altering the original tree-sitter nodes before rendering with the base unparser.

User-defined conversion functions are also supported in this direction, for tree adjustments that go beyond built-in edit actions. A custom function in this case takes the form

```
func(self: Wrapper, index: int, **kwargs) -> int
```

where `self` is the wrapper and `index` is the position of the child node to be transformed. The return value indicates the position from which the wrapper should continue processing after the function completes.

3 Validation

To validate that GramVarKit can instantiate an AI-oriented grammar variant from a configuration file, we use SimPy[10] as a case study, encode its design in a configuration file, and use GramVarKit to generate the corresponding grammar, parser, and two unparsers. The resulting SimPy specification is expressed primarily through declarative grammar-edit rules, with only a small number of direction-specific rules and user-defined conversion functions.

We evaluate the generated toolkit on two Python corpora: the Python subset of CodeSearchNet and a subset of the Stack v2 dataset consisting of Python files from repositories with over 5000 GitHub stars and file sizes below 32KB. Each sample is converted from Python to SimPy and back to Python using the original parser, the generated target parser, and the two generated unparsers. We then compare the original and reconstructed parse trees, ignoring comments, line-continuation markers, and a small set of optional brackets, to measure structural consistency rather than exact surface-form identity. The implementation and SimPy configuration are available at <https://github.com/zhangcen456/GramVarKit>.

Table 1 reports the parse-tree consistency rate and the average time for one round-trip conversion measured with multiprocessing using 16 workers. For all evaluated examples whose original code could be parsed by the tree-sitter Python parser without error nodes, the reconstructed code retained structural consistency after round-trip conversion. These results show that GramVarKit can successfully instantiate the SimPy variant from configuration rules and preserve the syntactic information needed to reconstruct the original Python code.

We also conduct a Java validation using the same setup, providing additional evidence that GramVarKit extends beyond Python.

4 Conclusion and Future Work

This paper introduced GramVarKit, a tree-sitter-based toolkit generator for AI-oriented grammar variants derived from existing programming languages. For variants that preserve the original AST structure and can be described through localized grammar edits, GramVarKit compiles configuration-level transformation rules into the derived grammar and the corresponding bidirectional conversion toolchain, thereby significantly reducing the amount of manual efforts. Using SimPy as a case study, we showed that GramVarKit can instantiate a concrete grammar variant from configuration rules and preserve the syntactic information needed to reconstruct the original Python code through round-trip conversion.

Beyond reducing per-variant engineering cost, GramVarKit may also support more systematic exploration of AI-oriented grammar variant design. Different grammar variants can be instantiated through different rule combinations, with their corresponding toolkits generated automatically. This enables researchers to convert existing code corpora and evaluate multiple grammar variants under a common workflow. This may further provide a basis for automated grammar-search or optimization techniques that generate, evaluate, and refine candidate variants according to objectives such as efficiency, correctness, and model performance.

Acknowledgments

This research/project is supported by the Australian Research Council under its Discovery Early Career Researcher Award (DECRA) funding scheme (DE260100192)

References

- [1] Anthropic. n.d.. *Claude Code*. Retrieved May 4, 2026 from <https://www.anthropic.com/product/claude-code>
- [2] ast-grep. [n. d.]. *ast-grep/ast-grep: A CLI tool for code structural search, lint and rewriting. Written in Rust*. Retrieved May 4, 2026 from <https://github.com/ast-grep/ast-grep>
- [3] Ruisheng Cao, Mouxian Chen, Jiawei Chen, Zeyu Cui, Yunlong Feng, Binyuan Hui, Yuheng Jing, Kaixin Li, Mingze Li, Junyang Lin, Zeyao Ma, Kashun Shum, Xuwu Wang, Jinxi Wei, Jiaxi Yang, Jiajun Zhang, Lei Zhang, Zongmeng Zhang, Wenting Zhao, and Fan Zhou. 2026. Qwen3-Coder-Next Technical Report. *arXiv preprint arXiv:2603.00729* (2026).
- [4] Matteo Ciccaglione, Pierciro Caliendo, and Alessandro Pellegrini. 2025. Tahr: The Generative Attribute Grammar Framework. *arXiv:2512.01872 [cs.FL]* <https://arxiv.org/abs/2512.01872>
- [5] comby-tools. [n. d.]. *comby-tools/comby: A code rewrite tool for structural search and replace that supports ~every language*. Retrieved May 4, 2026 from <https://github.com/comby-tools/comby>
- [6] Anton Lozhkov et al. 2024. StarCoder 2 and The Stack v2: The Next Generation. *arXiv:2402.19173 [cs.SE]* <https://arxiv.org/abs/2402.19173>
- [7] Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. 2019. CodeSearchNet challenge: Evaluating the state of semantic code search. *arXiv preprint arXiv:1909.09436* (2019).
- [8] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361* (2020).
- [9] OpenAI. 2025. *Introducing Codex*. Retrieved May 4, 2026 from <https://openai.com/index/introducing-codex/>
- [10] Zhensu Sun, Xiaoning Du, Zhou Yang, Li Li, and David Lo. 2024. AI Coders Are among Us: Rethinking Programming Language Grammar towards Efficient Code Generation. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 2024)*. Association for Computing Machinery, 1124–1136. doi:10.1145/3650212.3680347
- [11] tree-sitter. n.d.. *tree-sitter/tree-sitter: An incremental parsing system for programming tools*. Retrieved May 4, 2026 from <https://github.com/tree-sitter/tree-sitter>
- [12] Weixing Zhang, Jörg Holtmann, Daniel Strüder, Regina Hebig, and Jan-Philipp Steghöfer. 2024. Supporting Meta-model-based Language Evolution and Rapid Prototyping with Automated Grammar Optimization. *arXiv:2401.17351 [cs.SE]* <https://arxiv.org/abs/2401.17351>